MINI PROJECT REPORT

on

# Disguising AI-Generated Text as Human Written

Submitted by


| | |
|---|---|
| **J Ashwin Balagi** | 3122237001006 |
| **Gopika Ganesan** | 3122237001013 |
| **I S Rajesh** | 3122237001042 |
| **Siddharth M** | 3122237001049 |
| **Tarun Suresh** | 3122237001055 |


Under the guidance of

**Dr. Nilu R. Salim**


Department of Computer Science and Engineering
SSN College of Engineering, Kalavakkam, Chennai


**2025**

# Declaration

We hereby declare that the mini project titled ***"Disguising AI-Generated Text as Human Written"*** is submitted by us in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering. The work presented in this report is original and has not been submitted elsewhere for the award of any qualification.

| | |
|---|---|
| J Ashwin Balagi | Gopika Ganesan |
| I S Rajesh | Siddharth M |
| Tarun Suresh | |

# Acknowledgement

We would like to express our sincere gratitude to our guide for their continuous support and guidance. We also thank our institution for providing the resources required to complete this project.

# Abstract

With the rapid advancement of large language models, distinguishing AI-generated text from human-written content has become increasingly challenging. The CLEF ELOQUENT Voight-Kampff task focuses on generating text that closely mimics human writing while evading automated detection systems.

In this project, we conduct a comparative analysis of four techniques aimed at improving the human-likeness of AI-generated text: Retrieval-Augmented Generation (RAG), adversarial search guided by a RoBERTa detector, GAN-based refinement using an LLM discriminator, and back-translation via Gemini. Each method is implemented and evaluated independently to assess its effectiveness in reducing detectability and enhancing naturalness.

**Keywords:** AI-generated text, human-likeness, adversarial search, RAG, back-translation, GAN refinement, CLEF ELOQUENT, Voight-Kampff.

# Contents

**Declaration** **1**

**Acknowledgement** **2**

**Abstract** **3**

**1 Introduction** **6**

1.1 Background . . . . . 6

1.2 Problem Statement . . . . . 6

1.3 Objectives . . . . . 6

**2 Literature Survey** **7**

**3 Methodology** **8**

3.1 System Architecture . . . . . 8

3.2 Dataset . . . . . 9

3.3 Base Text Generation . . . . . 9

3.4 Adversarial Search . . . . . 9

3.5 Retrieval-Augmented Generation (RAG) . . . . . 10

3.6 Back-Translation . . . . . 11

3.7 GAN-Based Refinement . . . . . 12

**4 Implementation** **13**

4.1 Tools and Technologies . . . . . 13

4.2 Modules . . . . . 13

**5 Results and Discussion** **15**

5.1 Evaluation Metrics . . . . . 15

5.2 Sample Evaluation Results . . . . . 15

5.3 Evaluation Report Screenshots . . . . . 16

5.4 Discussion by Technique . . . . . 16

5.4.1 Adversarial Search . . . . . 16

5.4.2 RAG . . . . . 16

5.4.3 Back-Translation . . . . . 16
5.4.4 GAN Refinement . . . . . 17
5.5 RAG Detector Score Reduction . . . . . 17
5.6 Sample Generated Output . . . . . 17

**6 Conclusion and Future Work 18**
6.1 Conclusion . . . . . 18
6.2 Future Work . . . . . 18

# Chapter 1

# Introduction

## 1.1 Background

Recent advances in large language models have enabled the generation of highly fluent and coherent text across a wide range of domains. Despite their fluency, these models often exhibit subtle statistical patterns and artifacts that distinguish them from human-written text. The CLEF ELOQUENT Voight-Kampff task evaluates how effectively generated content can evade automated detection systems while maintaining naturalness and coherence.

## 1.2 Problem Statement

To evaluate and compare different techniques that can transform or refine AI-generated text to make it more human-like and less detectable by automated detection systems.

## 1.3 Objectives

- Generate diverse AI-based textual outputs using the `Eloquent/Voight-Kampff` dataset.
- Implement and evaluate four techniques: Adversarial Search, GAN-based Refinement, RAG, and Back-Translation.
- Compare approaches based on their ability to improve human-likeness and reduce detectability.

# Chapter 2

# Literature Survey

Recent research has focused on distinguishing machine-generated content from human writing while improving generation quality. Detection systems exploit statistical patterns such as token-level predictability and low perplexity to identify AI-generated content.

Table 2.1: Summary of Related Work

| **Work** | **Approach** | **Key Contribution** | **Venue** |
|---|---|---|---|
| CLEF ELOQUENT [1] | Voight-Kampff evaluation | Benchmark for human-vs-AI text | CLEF 2024 |
| TuringBench [2] | Turing-test benchmark | 19-class AI-text dataset | EMNLP 2021 |
| GAN [3] | Generative Adversarial Nets | Foundation of adversarial refinement | NeurIPS 2014 |
| BERT [4] | Bidirectional Transformer | Discriminator backbone | NAACL 2019 |
| RAG [5] | Knowledge-grounded gen. | Retrieval reduces hallucination | NeurIPS 2020 |
| Sentence-BERT [7] | Siamese BERT embeddings | Semantic retrieval for RAG | EMNLP 2019 |
| FAISS [8] | Nearest-neighbor search | Efficient corpus indexing | Meta AI 2017 |

# Chapter 3

# Methodology

## 3.1 System Architecture

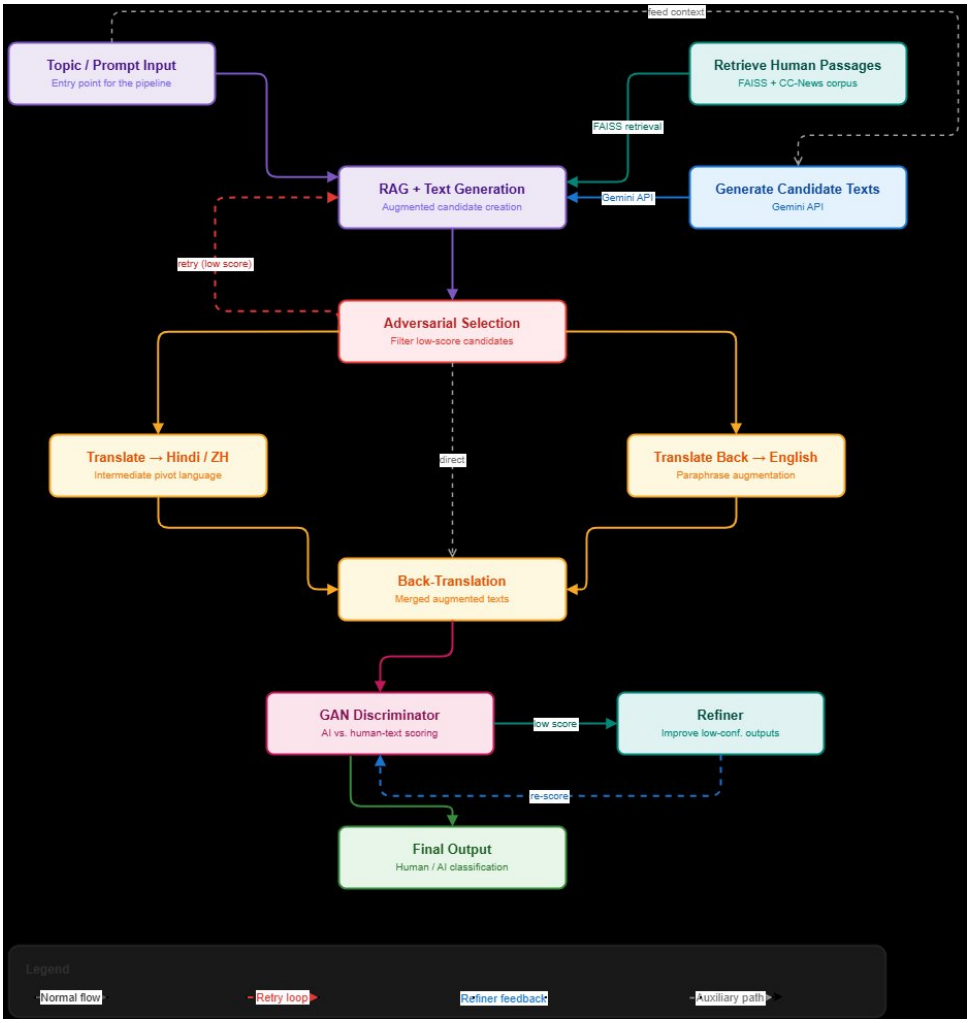


Figure 3.1: Overall System Architecture

The pipeline loads topics from the `Eloquent/Voight-Kampff` test dataset. Each topic provides a base prompt, genre/style specification, and content requirements. Text is generated using Mistral-7B and passed through one of four humanization techniques

before submission as `NNN.txt` files zipped for upload.

## 3.2 Dataset

All approaches use the `Eloquent/Voight-Kampff` HuggingFace dataset (`test` split). Each row contains a `voight-kampfftesttopics` field with a suggested prompt and topics, each having an `id`, `Genre and Style`, and `Content` field.

## 3.3 Base Text Generation

All approaches use `mistralai/Mistral-7B-Instruct-v0.2` loaded via the HuggingFace `pipeline` with `device_map="auto"` and `max_new_tokens=600`. Prompts are constructed by combining the dataset's suggested prompt with the topic's genre and content fields.

```
generator = pipeline(
    "text-generation",
    model="mistralai/Mistral-7B-Instruct-v0.2",
    device_map="auto", max_new_tokens=600
)

def build_prompt(topic, suggested_prompt):
    genre_line  = "Genre and Style: " + topic.get("Genre and Style",
        "")
    content_str = topic["Content"]
    return suggested_prompt + "\n" + genre_line + "\nItems:\n" +
        content_str

messages = [{"role": "user", "content": full_prompt}]
result   = generator(messages, do_sample=True, temperature=0.8)
text     = result[0]["generated_text"][-1]["content"].strip()
```

Listing 3.1: Base Prompt Construction and Generation

## 3.4 Adversarial Search

Uses `roberta-base-openai-detector` to score each candidate. Three texts are generated per topic; the one with the lowest detector score (hardest to detect as AI) is selected. An iterative refinement loop regenerates up to 3 times if the score does not fall below a threshold of 0.35.

```
detector_model = AutoModelForSequenceClassification.from_pretrained(
    "roberta-base-openai-detector").to(DEVICE)

def detector_score(text):
```

```
    inputs = detector_tokenizer(text, return_tensors="pt",
                 truncation=True, max_length=512).to(DEVICE)
    with torch.no_grad():
        logits = detector_model(**inputs).logits
    return torch.softmax(logits, dim=1)[0][1].item()

def generate_best_candidate(prompt, num_candidates=3):
    candidates = [(generate_text(prompt), 0) for _ in
       range(num_candidates)]
    candidates = [(t, detector_score(t)) for t, _ in candidates]
    candidates.sort(key=lambda x: x[1])
    return candidates[0]   # lowest score = hardest to detect

def adversarial_refine(prompt, threshold=0.35, max_iters=3):
    best_text, best_score = None, 1.0
    for _ in range(max_iters):
        text  = generate_text(prompt)
        score = detector_score(text)
        if score < best_score:
            best_score, best_text = score, text
        if score < threshold:
            break
    return best_text
```

Listing 3.2: Adversarial Candidate Selection and Refinement

## 3.5 Retrieval-Augmented Generation (RAG)

Retrieves real human-written passages from the CC-News corpus (first 500 documents, >200 words each, truncated to 1500 characters). Documents are encoded using `sentence-transformers` and indexed with FAISS `IndexFlatL2`. The top-3 most similar passages are prepended to the prompt as style references.

```
embedder    =
   SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
wiki        = load_dataset("cc_news", split="train[:500]")
documents   = [row["text"][:1500] for row in wiki
               if len(row["text"].split()) > 200]
embeddings  = embedder.encode(documents, convert_to_numpy=True)
index       = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)

def retrieve_context(query, k=3):
    q_emb = embedder.encode([query])
    _, idx = index.search(q_emb, k)
    return [documents[i] for i in idx[0]]
```

```python

def build_rag_prompt(base_prompt, genre, content):
    passages = retrieve_context(base_prompt + " " + genre, k=3)
    context  = "\n\n".join(passages)
    return (f"Below are examples of human writing style:\n\n{context}\n\n---\n\n"
            f"Now write a new original text.\n\n{base_prompt}\n\n"
            f"Genre: {genre}\n\nContent:\n{content}\n\n"
            f"Write ~500 words. Do not copy examples.")
```

Listing 3.3: RAG: FAISS Indexing and Retrieval

Detection uses a curvature approximation: log-probability of the original text minus log-probability after a minimal perturbation (adding a space after each period).

## 3.6 Back-Translation

Applies post-processing using the Gemini API (`gemini-2.5-flash`). Each generated `.txt` file is read, a pivot language (Hindi or Chinese) is randomly chosen, and Gemini translates the text into the pivot and back to English with a "literal" re-translation prompt requesting casual human phrasing and varied sentence structure.

```python
import google.generativeai as genai, random

genai.configure(api_key=GEMINI_API_KEY)
gemini_model = genai.GenerativeModel("gemini-2.5-flash")
LIF_LANGS    = ["Hindi", "Chinese"]

def back_translate_file(file_path):
    original_text = file_path.read_text(encoding="utf-8")
    pivot         = random.choice(LIF_LANGS)
    refine_prompt = (
        f"I have this text: '{original_text}'\n\n"
        f"Please translate this into {pivot}, then translate it back "
        "into English. The final English version should be a 'literal' "
        "translation. Keep it slightly raw, use casual human phrasing, "
        "and vary the sentence structure. Do not use AI formatting like "
        "bullet points or 'In conclusion'."
    )
    response = gemini_model.generate_content(refine_prompt)
    file_path.write_text(response.text.strip(), encoding="utf-8")
    return pivot
```

Listing 3.4: Back-Translation via Gemini API

## 3.7 GAN-Based Refinement

Implements a three-component adversarial loop via OpenRouter API: a **Generator**, **Discriminator**, and **Refiner**. Text is scored on a 0–10 humanness scale; outputs below 8.0 are rewritten using structured JSON feedback (up to 3 rounds). A post-processing step expands contractions to reduce surface-level AI artifacts.

The Generator uses a detailed system prompt requiring sensory details, varied sentence rhythm, mid-thought openings, and self-corrections (temperature 0.92). The Discriminator returns a JSON object with score, verdict, AI tells, missing human qualities, and a rewrite instruction. The Refiner rewrites using that critique (temperature randomised 0.85–0.97 per round).

```
HUMANNESS_THRESHOLD = 8.0
MAX_REFINE_ROUNDS   = 3

def gan_generate(topic, suggested_prompt):
    text = generate(topic, suggested_prompt, temperature=0.92)
    for round_num in range(MAX_REFINE_ROUNDS + 1):
        critique = discriminate(text)          # returns JSON
        score    = float(critique.get("score", 5.0))
        if score >= HUMANNESS_THRESHOLD or round_num ==
            MAX_REFINE_ROUNDS:
             break
        text = refine(text, critique, topic)   # rewrites using
            feedback
    return {"final_text": humanize(text), "best_score": score}

CONTRACTIONS = [
    (r"\bdo not\b", "don't"), (r"\bcannot\b",  "can't"),
    (r"\bI am\b",   "I'm"),   (r"\bit is\b",   "it's"),
]
def humanize(text):
    for pattern, replacement in CONTRACTIONS:
        text = re.sub(pattern, replacement, text,
            flags=re.IGNORECASE)
    return text
```

Listing 3.5: GAN Loop: Generator, Discriminator, Refiner

# Chapter 4

# Implementation

## 4.1 Tools and Technologies

Table 4.1: Tools and Technologies Used

| **Tool / Library** | **Purpose** |
|---|---|
| Python 3.10+ | Primary programming language |
| Mistral-7B-Instruct-v0.2 | Base text generation (all approaches) |
| HuggingFace Transformers | Model loading, generation, detection pipelines |
| `roberta-base-openai-detector` | AI text detector for adversarial scoring |
| SentenceTransformers (all-MiniLM-L6-v2) | Embeddings for RAG retrieval |
| FAISS (IndexFlatL2) | Vector similarity search for RAG |
| CC-News dataset | Human-written retrieval corpus for RAG |
| Gemini API (gemini-2.5-flash) | Back-translation pivot processing |
| OpenRouter API | LLM backend for GAN components |
| HuggingFace Datasets | Loading `Eloquent/Voight-Kampff` dataset |

## 4.2 Modules

1. **Base Generation** — Loads Mistral-7B, builds prompts from dataset fields, generates ~500-word texts per topic.

2. **Adversarial Selection** — Generates 3 candidates, scores each with `roberta-base-openai-detecto` selects lowest-scoring, refines up to 3 iterations (threshold 0.35).

3. **RAG Module** — Builds FAISS index over 500 CC-News documents, retrieves top-3 passages per query, injects as style references.
4. **Back-Translation** — Applies Gemini-based pivot translation (Hindi or Chinese) as post-processing on each `.txt` file.
5. **GAN Module** — Runs Generator-Discriminator-Refiner loop via OpenRouter; applies contraction expansion post-processing.

# Chapter 5

# Results and Discussion

## 5.1 Evaluation Metrics

Table 5.1: Evaluation Metrics and Interpretation

| **Metric** | **Description** | **Human-Like Value** |
|---|---|---|
| Binoculars Score | Ratio-based detector using two LLMs | Higher (≈1.0) |
| GPT-2 Perplexity | Language model uncertainty on text | Higher |
| Avg Token Rank | Mean rank of predicted tokens | Higher |
| RoBERTa Confidence | Classifier probability of "Real" | Higher |

## 5.2 Sample Evaluation Results

Table 5.2: Evaluation Scores for Sample Outputs

| **Technique** | **Binoculars** | **GPT-2 Perp.** | **Avg Token Rank** | **RoBERTa** |
|---|---|---|---|---|
| Adversarial Search | 0.9913 | 72.65 | 363.57 | 0.9995 (Real) |
| GAN Refinement | 0.9997 | 228.66 | 412.71 | 0.3514 (Fake) |

## 5.3 Evaluation Report Screenshots

(a) Adversarial Search — **LIKELY HUMAN** (Composite: 1.5757)

(b) GAN Refinement — **POSSIBLE AI** (Composite: 1.4593)

Figure 5.1: Terminal Evaluation Reports

## 5.4 Discussion by Technique

### 5.4.1 Adversarial Search

- Generates 3 candidates per topic; selects lowest `roberta-base-openai-detector` score.
- Iterative refinement (up to 3 rounds, threshold 0.35) further reduces detector confidence.
- Achieves RoBERTa confidence of 0.9995 (Real) on the sample output.

### 5.4.2 RAG

- Retrieval from CC-News reduces artificial structure in generated text.
- Relative reduction in detectability: **81.8%** (baseline 0.0269 vs. RAG 0.0049).
- Retrieval quality depends on the closeness of the query to the CC-News domain.

### 5.4.3 Back-Translation

- Pivot language (Hindi or Chinese) randomly chosen per document.
- Gemini prompted for "literal" re-translation with raw, casual phrasing.

- Disrupts repetitive AI-specific phrasing and rigid syntactic patterns.

#### 5.4.4 GAN Refinement

- Three-component loop with structured JSON critique from Discriminator.
- Highest GPT-2 perplexity (228.66) and avg token rank (412.71).
- RoBERTa flags output as Fake (0.6486), suggesting the technique optimises for statistical unpredictability over detector-specific patterns.

## 5.5 RAG Detector Score Reduction

Table 5.3: RAG vs. Baseline Detector Score

| **Condition** | **Detector Score** | **Label** | **Reduction** |
|---|---|---|---|
| Without RAG (Baseline) | 0.0269 | AI-Generated | — |
| With RAG | 0.0049 | Human-Like | **81.8%** |

## 5.6 Sample Generated Output

The following passage was produced by the Adversarial Search pipeline and received the verdict **LIKELY HUMAN** (Composite Score: 1.5757).

> The air in the quiet corners of St. Luke's felt heavier that evening, thick with the scent of old parchment and the faintest hint of unshed ink. It was late, the kind of hour when shadows stretched long and the silence pressed close like a secret. Holmes sat by the hearth, his trench coat curling around his frame, the soft glow of a candle flickering over his weathered face. Outside, the city hummed with restless energy, but within the library's shadowed alcoves, the world narrowed to the weight of a single piece—an unmarked paper, its edges slightly frayed, bearing a secret that could unravel everything. Dr. Watson leaned forward, his voice low, almost a whisper. "If that passage is supposed to stay hidden, why would it end up here?" Soames, the younger tutor, sat across from him, his hands twisting the hem of his shirt, eyes darting between the book and the floor. "It must have been something urgent," he murmured, as if recalling a memory too recent to name. Holmes raised an eyebrow, the gesture careful, measured. "An incident of that scale requires a mind as sharp as a scalpel," he replied, his tone deliberate, almost conversational.

# Chapter 6

# Conclusion and Future Work

## 6.1 Conclusion

This project presents a comparative evaluation of four techniques aimed at improving the human-likeness of AI-generated text: Retrieval-Augmented Generation, Adversarial Search, GAN-based Refinement, and Back-Translation.

The results demonstrate that each technique contributes differently to enhancing text quality and reducing detectability. Adversarial search is highly effective in minimising detector confidence through iterative candidate selection and refinement, achieving a RoBERTa Real confidence of 0.9995. RAG improves contextual grounding by injecting real human-written passages, achieving an 81.8% reduction in detector score. Back-translation via Gemini introduces linguistic variability through pivot language round-trips. GAN-based refinement achieves the strongest statistical human-likeness (highest perplexity and token rank) through structured LLM critique but shows inconsistent results against fine-tuned detectors.

Overall, no single method is universally optimal. The effectiveness of each approach depends on the desired balance between coherence, diversity, and resistance to detection.

## 6.2 Future Work

- Adaptive evaluation strategies based on topic or genre.
- Fine-tuning discriminator models for improved accuracy.
- Incorporation of reinforcement learning for iterative refinement.
- Exploration of additional linguistic transformations for further improvement.

**GitHub Repository:** `https://github.com/ISRajesh183/NLP-MiniProject-CLEF`

# Bibliography


[1] CLEF ELOQUENT Lab, *Voight-Kampff Generated Text Detection Task*, CLEF, 2024. https://ceur-ws.org/Vol-3740/paper-225.pdf

[2] A. Uchendu, T. Le, and D. Lee, *TuringBench: A Benchmark Environment for Turing Test in the Age of Neural Text Generation*, EMNLP Findings, 2021.

[3] I. Goodfellow et al., *Generative Adversarial Networks*, NeurIPS, 2014.

[4] J. Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, NAACL, 2019.

[5] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS, 2020.

[6] Google DeepMind, *Gemini: A Family of Highly Capable Multimodal Models*, 2023.

[7] N. Reimers and I. Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, EMNLP, 2019.

[8] FAISS, *Facebook AI Similarity Search*, Meta AI, 2017.